Tetris Code Analysis

https://eae-git.eng.utah.edu/u1537288/tetris/-/tree/main

What software architecture did you use?

My Tetris project uses a component-based and data-driven architecture built on Unity's MonoBehaviour system. The Gameplay is divided between 2 main classes rather than one large controller. The Board class acts as the central authority for the game, managing the tilemap, validating positions, spawning pieces, clearing lines, and handling global effects such as bomb explosions. The Piece class is responsible for the behavior of the currently active piece, including movement, rotation, gravity timing, lock delay, and special behaviors while the piece is falling. By using a static data class, I can set up the layout of pieces that are paired (Dictionary) to a struct that uses an enumerated set of defined values. The data is a matrix based on the Tetris Super Rotation System (SRS). That helps calculate rotation values, especially those that aren't centered on their shapes, by applying a small offset value. UI behavior, such as toggling the controls panel and exiting the game, is handled by a small dedicated script so it does not interfere with core gameplay systems. I was considering adding an Audio manager and dedicated script as well; however, I am only using a small handful of sounds, and don't anticipate scaling up this project for it to be necessary.

```
public enum Tetromino
{
    I, J, L, O, S, T, Z,
    Bomb, Rainbow
}

[System.Serializable]
7 references
public struct TetrominoData
{
    public Tetromino tetromino;
    public Tile tile;

    public Tile[] specialTiles;
```

```
public static readonly Dictionary<Tetromino, Vector2Int[]> Cells = new Dictionary<Tet
{
    { Tetromino.I, new Vector2Int[] { new Vector2Int(-1, 1), new Vector2Int( 0, 1), n
    { Tetromino.J, new Vector2Int[] { new Vector2Int(-1, 1), new Vector2Int(-1, 0), n
    { Tetromino.L, new Vector2Int[] { new Vector2Int( 1, 1), new Vector2Int(-1, 0), n
    { Tetromino.O, new Vector2Int[] { new Vector2Int( 0, 1), new Vector2Int( 1, 1), n
    { Tetromino.S, new Vector2Int[] { new Vector2Int( 0, 1), new Vector2Int( 1, 1), n
    { Tetromino.T, new Vector2Int[] { new Vector2Int( 0, 1), new Vector2Int(-1, 0), n
```

Why did you choose this method?

I chose this method because the scope was relatively small and worked efficiently with Unity's design philosophy. Once the main data components were set up, it made setting up the grid-based board and pieces easy to incrementally add, as well as altering/adding to make adding new pieces seamless. Initially, a lot of values were hardcoded, but due to how Unity compiles scripts and allows you to expose values in the editor, testing and iterating were much easier. I think Unity works better for rapid prototyping, as compile times are much shorter, and exposing/accessing values and editing them while running is much more straightforward than Unreal. Not to mention, Tetris is 2D, which Unreal doesn't natively support like Unity.

In what ways was it good?

This architecture made the project flexible and extensible. Adding new mechanics rarely required modifying existing systems in large ways. Debugging was easier because each class had a clear responsibility, and issues could be traced to a specific system. The data-driven design reduced duplication and made the codebase easier to maintain. Once the core scripts were in place, it allowed for quick and simple additions for other features, like adding the ghost/piece projection effect to see where your block will land. The ease of talking to and referencing other classes made connecting, separating, and separating/organizing the codebase very streamlined.

In what ways was it bad?

Before adding the twist, some values were hard-coded, like the number of tiles per piece, and were not initially equipped to add special blocks (at least in the struct). To combat this, a dedicated field for special pieces was added. While iterating initially, I coded it to support a specific type of special piece (the bomb), but later made it more general and added certain conditionals so it was possible to toggle certain parameters (everything was inherited, but making it more modular allows for combining certain mechanics from blocks, possible without changing the core structure in the future).

```
1 reference
public bool HasSpecialTiles =>
    specialTiles != null && specialTiles.Length > 0;
```

```
public void Set(Piece piece)
{
    for (int i = 0; i < piece.cells.Length; i++)
    {
        Vector3Int pos = piece.position + piece.cells[i];

        if (piece.data.HasSpecialTiles)
        {
            int index = (i + piece.colorIndex) % piece.data.specialTiles.Length;
            tilemap.SetTile(pos, piece.data.specialTiles[index]);
        }
        else
        {
            tilemap.SetTile(pos, piece.data.tile);
        }

    }
```

```
cells = new Vector3Int[4];
```

```
cells = null;
```

```
if (cells == null || cells.Length != trackingPiece.cells.Length)
{
    cells = new Vector3Int[trackingPiece.cells.Length];
}
```

The piece projection initially was hard-coded to 4, which gave my 3-piece block an extra tile, but having it be dynamically allocated allowed it to determine the piece tile amount based on the data.

What did you learn from doing it this way?

Even small systems need to remove hidden assumptions in design. While developing the system, harmless assumptions about all parts being the same size or behaving in a specific way turn into limitations later on when new features like bombs or rainbow pieces are added. I learned that it is important to treat data and functions separately. This approach allows new features to be added easily without changing the existing code. I also realized that good design is more about making future changes easier instead of just lowering complexity.

How would you do it differently if you had to do it again?

If I were to redo the project, I would plan for special pieces and variable sizes earlier, and I would separate input handling into its own system. I would also reduce conditional logic by using more explicit behavior flags or lightweight behavior components. Overall, I gained valuable knowledge on class/code structure and organization, and how to build a scalable project when starting from scratch.

Other Thoughts

Part of the time spent on this project was dedicated to researching how the original Tetris game actually works beyond the surface-level mechanics. This included not only obvious features such as hard drops and piece randomization, but also the underlying systems that support them, like how piece shapes and rotations are defined in data and how wall kick rules prevent pieces from

rotating outside the board. These behind-the-scenes mechanics are largely invisible to players, but they are critical to how the game feels and functions. Studying these systems helped me better understand how much technical structure is required to support what appears to be a simple game.

Time Spent

This Project took about **11 hours** to complete. From ideation to final, which includes researching the original game (or the versions I found online), dissecting the mechanics and underlying systems, and reading the wiki and other resources about how the game works. Then building the actual game and figuring out the structure (like the enum and data), and then building the main classes (pieces and board) to take care of game states and what is essentially the player character (the piece class). https://tetris.fandom.com/wiki/Super_Rotation_System This article helped tremendously in setting up the data for piece shape, wall kicks, and rotation behavior.